

Lecture 16: CNN Foundations

One neuron, reused at every pixel

Parts of this chapter adapt LMU I2DL (slds-lmu), CC BY 4.0.

A pomegranate-sorting line

A camera photographs each fruit on a conveyor;
a model grades it - **export**, **market**, or **juice**.

This sorting line runs through all three lectures.

Your ch11 homework MLP (784-256-128-10)
already had 235,146 weights - for a 28×28
grayscale thumbnail. The camera's photo is
 $224 \times 224 \times 3 = 150,528$ numbers.

Predict: how many weights does *one* dense layer
with 1,000 hidden units need for that photo?

A pomegranate-sorting line

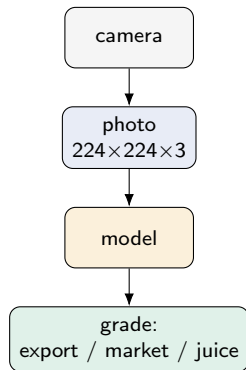
A camera photographs each fruit on a conveyor;
a model grades it - **export**, **market**, or **juice**.

This sorting line runs through all three lectures.

Your ch11 homework MLP (784-256-128-10)
already had 235,146 weights - for a 28×28
grayscale thumbnail. The camera's photo is
 $224 \times 224 \times 3 = 150,528$ numbers.

Predict: how many weights does *one* dense layer
with 1,000 hidden units need for that photo?

$150,528 \times 1,000 \approx$ **150 million** - for *one* layer, be-
fore learning anything. Today's fix costs a few *hun-*
dred.



Outline

Why dense nets are the wrong tool for images

The convolution operation

Why convolution is the right prior

The components of a conv layer

Variations on the kernel

Your first CNN

Images are not feature vectors

A dense net treats a photo as a bag of unrelated numbers. That throws away the one thing images have: structure.

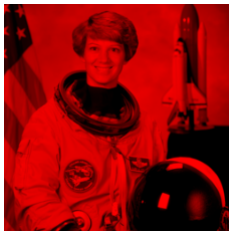
Color, recalled from [33]

[33] did the physics: three cone types, so a pixel is **three numbers** (R, G, B), each 0 to 255 (HSV re-coordinatizes the same pixel - it returns in HW1b). Today only the **shape** matters: a color photo is **three stacked grids**, one per channel - a $3 \times H \times W$ block of numbers.

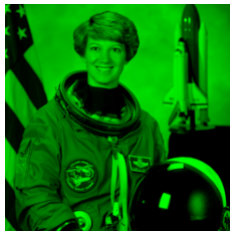
RGB image



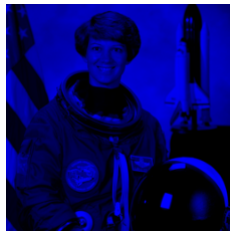
Every pixel is three numbers: (R, G, B), each 0-255
R channel



G channel



B channel



Hold on to "stacked grids". In Section 4 the stack depth becomes a conv layer's **input channels** - and every conv layer then builds a stack of its own.

An image is a grid of numbers

A grayscale image is **one matrix**: each entry is a pixel intensity, 0 (black) to 255 (white).

RGB or HSV, color is three stacked grids. From here we work in **grayscale** - one matrix - so the convolution mechanics stay 2D; channels return in Section 4.

To a computer there is no "image" - only a grid of numbers.

A grayscale image



8x8 patch: just numbers, 0-255

189	144	142	171	169	124	84	38
118	109	113	113	112	89	48	40
99	101	104	97	73	39	39	37
80	37	69	81	49	33	32	32
24	38	59	58	33	31	30	29
32	39	57	45	35	36	26	18
30	40	63	41	42	31	30	75
29	37	44	42	42	39	96	103

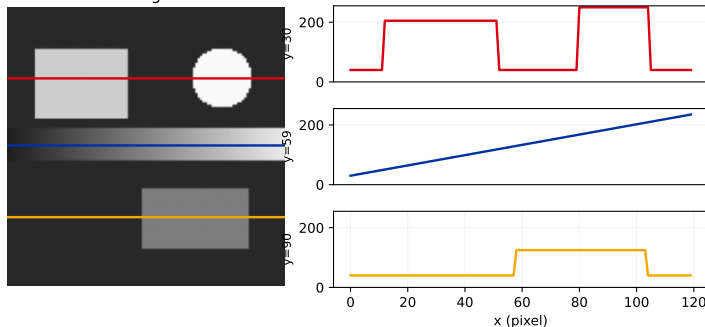
An image is a function

Read the grid as a **function**
 $f(x, y)$: position in,
intensity out.

Walk along one horizontal
line: f is flat on uniform
regions, a **ramp** across a
gradient, and a **jump** at an
edge.

An edge is where f changes
fast. That single idea is how
we will detect edges.

An image is a function: intensity $f(x)$ along each scan line
image



An MLP cannot see structure

Flattening a photo into a 150,528-vector hands the pixels to the net in *some* order - but a dense layer treats every order the same.

Shuffle the pixels with one fixed permutation and an MLP learns it *exactly as well*. To us, one is a pomegranate and the other is noise.

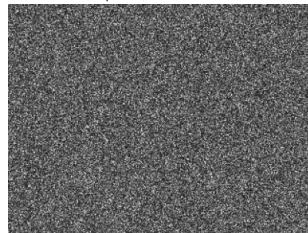
Nearby pixels are related; far-apart ones usually are not. A dense layer knows none of this.

A flatten-then-MLP sees these two as equally learnable

Original



Same pixels, one fixed shuffle



Two wishes

We want a layer that builds in what we know about images:

1. **Locality.** Detect a small pattern (an edge, a corner) from a small neighborhood of pixels - not from the whole image at once.
2. **Reuse.** A detector that is useful in one corner is useful everywhere. A cat in the corner is still a cat.

Wish 2 buys a property with a technical name, **translation equivariance**: shift the input, and the detector's output map shifts by the same amount. In 1D, with kernel $(1, 2, 3)$: input $(0, 0, 1, 0, 0) \rightarrow (3, 2, 1)$; move the spike one step right, $(0, 0, 0, 1, 0) \rightarrow (0, 3, 2)$ - the same response, one step later.

These two wishes - **locality** and **weight reuse** - are the convolutional layer. The rest of this lecture is just spelling them out.

Nature had the same two wishes

1959-62: **David Hubel** and **Torsten Wiesel** recorded single neurons in a cat's visual cortex while showing it bars of light (Nobel Prize, 1981). Two kinds of cell turned up:

Cell	Fires when...	In a CNN
simple	an edge at <i>one orientation</i> appears in <i>one small patch</i> of the visual field	a conv kernel (local, oriented - a Sobel)
complex	that same edge appears <i>anywhere</i> in a somewhat larger region	pooling over nearby detectors

The brain tiles the whole visual field with copies of the same small detectors - locality *and* reuse.

Convolution and pooling were not invented from scratch: they are an engineer's copy of the first two stages of the visual cortex. The next slide traces that copy.

Where CNNs came from

Convolutional nets are not new - three steps took them from biology to working code:

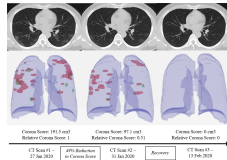
- ▶ **1980 - Kunihiro Fukushima's neocognitron.** Layers of "S-cells" and "C-cells", named after Hubel and Wiesel's simple and complex cells: convolution and pooling, stacked. But no backprop - its detectors were not trained end to end.
- ▶ **1989 - Yann LeCun** and colleagues at Bell Labs train the same kind of layers **with backpropagation** to read **handwritten digits** - the ZIP codes on U.S. mail.
- ▶ **1998 - LeNet-5** and the **MNIST** digit benchmark (LeCun, Bottou, Bengio & Haffner) become the reference point for a decade. Everything in this lecture descends from it.

Same recipe then and now: local filters, shared weights, pooling, a dense head. Later lectures just make it bigger, deeper, and trained on far more data.

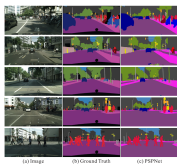
CNNs in the wild



self-driving perception



medical CT screening



semantic segmentation



image colorization

All built from this chapter's machinery. Images: LMU I2DL (slds-lmu), CC BY 4.0.

The convolution operation

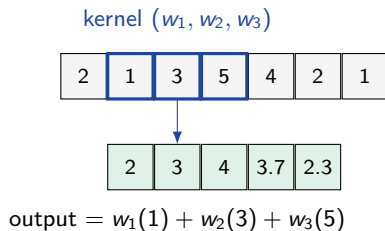
Slide a small window across the input; at each stop, take a dot product.

Start in 1D: a sliding dot product

Take a length-3 kernel $\mathbf{w} = (w_1, w_2, w_3)$ and slide it along a sequence. Each output is the dot product of the kernel with the window under it.

With $\mathbf{w} = (\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$ this is exactly a **moving average**. Different weights detect different local patterns.

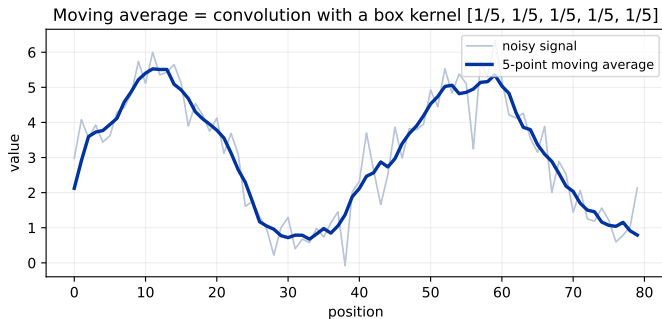
Same idea works on any sequence - audio, sensor traces, text. Convolution is a prior, not an image trick (more in L18).



The moving average is a convolution

Slide a box kernel $[\frac{1}{5}, \frac{1}{5}, \frac{1}{5}, \frac{1}{5}, \frac{1}{5}]$ over a noisy signal: each output is the **average** of its neighbours, so the signal comes out **smoothed**.

Blur, smooth, denoise - all the same sliding-and-summing. Different weights give different effects.



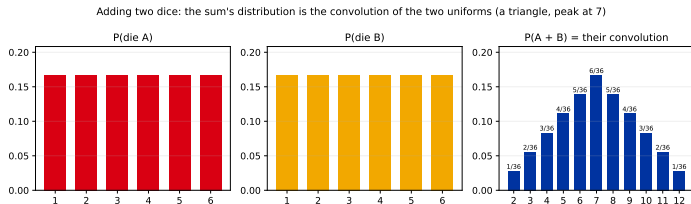
Aside: convolution lives in probability too

Roll **two dice** and add them.

The distribution of the *sum* is the **convolution** of the two single-die distributions.

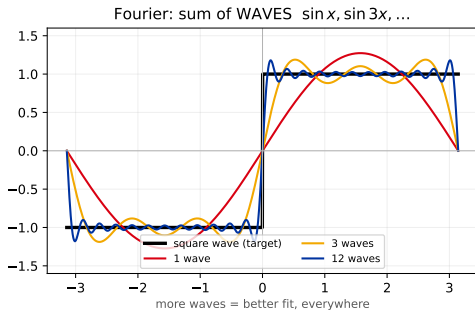
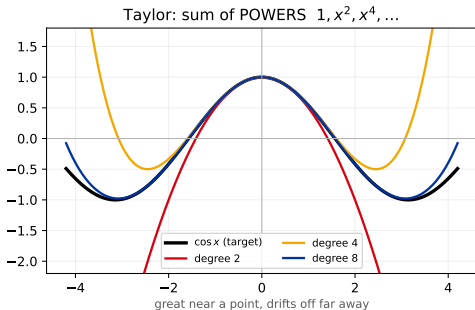
Two flat uniforms slide across each other and their overlap is summed - giving the triangle that peaks at 7.

Same operation, different field.
Convolution is a general tool,
not an image trick.



Aside: from Taylor to Fourier

Same idea, different building blocks: break a function into simple pieces

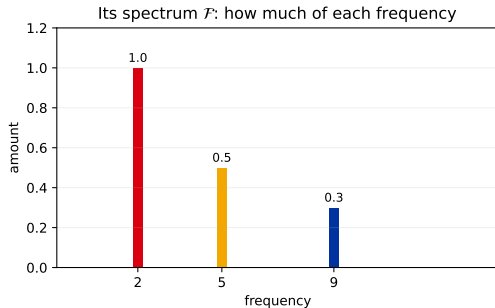
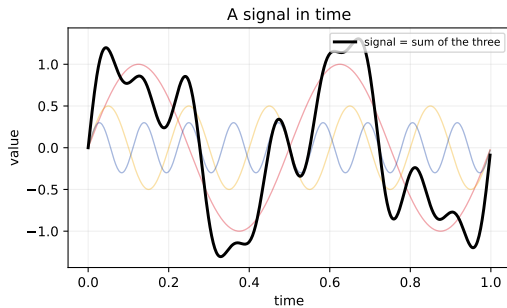


You already know **Taylor**: rebuild a function as a sum of **powers** $1, x, x^2, \dots$ - a great fit near a point. **Fourier** is the same move with different blocks: a sum of **waves** $\sin x, \sin 2x, \dots$ of rising *frequency*, and it fits everywhere.

Both break a function into simple pieces. Taylor's are powers; Fourier's are labelled by **frequency**.

A signal is a bundle of frequencies

The Fourier transform reads off the frequencies hiding in a signal



Add three sine waves and you get one wiggly signal. The **Fourier transform** $\mathcal{F}$ runs it backwards: it reads off *how much of each frequency* sits inside - three tones in, three clean spikes out. That is the signal's **spectrum**.

Same information, two views: the wave in time, or its recipe in frequency.

Aside: convolution lives in frequency too

That detour buys one clean fact - the **convolution theorem**:

$$\mathcal{F}\{f * g\} = \mathcal{F}\{f\} \cdot \mathcal{F}\{g\}$$

Convolving in space = multiplying in frequency. The messy slide-and-sum turns into a plain pointwise product once you switch to the frequency view.

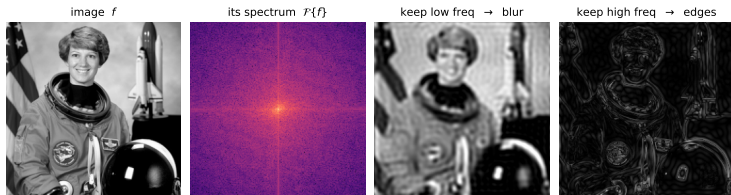
So the fast way to convolve a large image with a large kernel is to FFT both, multiply, and transform back:

$$f * g = \mathcal{F}^{-1}\{ \mathcal{F}\{f\} \cdot \mathcal{F}\{g\} \}.$$

A kernel is a **frequency filter**: it scales each frequency of the image up or down.

Blur and edges are frequency filters

Send the image to frequency space - its **spectrum** (panel 2: bright centre = low frequencies = smooth areas; outer = high = fine detail and edges).



Keep the low frequencies and you get a **blur**. **Keep the high** ones and you are left with **edges**.

You are about to meet these as kernels: the **box blur** is a low-pass filter, and **Sobel** is a high-pass one.

2D convolution, by hand

Input I , kernel W (2×2). Slide W over I ; each output cell is the dot product with the 2×2 window under it.

$$I = \begin{bmatrix} 1 & 2 & 0 \\ 3 & 1 & 1 \\ 0 & 2 & 4 \end{bmatrix} \quad W = \begin{bmatrix} 0 & 1 \\ 2 & 2 \end{bmatrix}$$

Top-left window $\begin{bmatrix} 1 & 2 \\ 3 & 1 \end{bmatrix}$:

$$s_{11} = 1 \cdot 0 + 2 \cdot 1 + 3 \cdot 2 + 1 \cdot 2 = 10$$

Repeat for all four positions:

$$S = \begin{bmatrix} 10 & 4 \\ 5 & 13 \end{bmatrix}$$

In one line:

$$s_{ij} = \sum_{m,n} I(i+m-1, j+n-1) w(m, n)$$

A 2×2 kernel on a 3×3 input gives a 2×2 output. The map *shrinks* - we fix that with padding in Section 4.

Convolution, step by step

Watch the kernel slide. At each stop the output cell is a **weighted average** of the window under it. **(Click to advance.)**

Here the kernel is a *Gaussian blur* (center weighted most, divided by 16), so the patch comes out *smoothed* on the same scale. Cells are shaded by value - light is high.

Convolution: slide, multiply, sum

step 1 of 9

input (5x5, shaded by value)

1	2	3	2	1
2	5	7	5	2
3	7	9	7	3
2	5	7	5	2
1	2	3	2	1

kernel: Gaussian blur / 16

1	2	1
2	4	2
1	2	1

output(0,0) =

weighted average of
the 9 window values

= 4.5

output (smoothed)

4.5		

Convolution, step by step

Watch the kernel slide. At each stop the output cell is a **weighted average** of the window under it. **(Click to advance.)**

Here the kernel is a *Gaussian blur* (center weighted most, divided by 16), so the patch comes out *smoothed* on the same scale. Cells are shaded by value - light is high.

Convolution: slide, multiply, sum

step 2 of 9

input (5x5, shaded by value)

1	2	3	2	1
2	5	7	5	2
3	7	9	7	3
2	5	7	5	2
1	2	3	2	1

kernel: Gaussian blur / 16

1	2	1
2	4	2
1	2	1

output(0,1) =

weighted average of
the 9 window values

= 5.6

output (smoothed)

4.5	5.6	

Convolution, step by step

Watch the kernel slide. At each stop the output cell is a **weighted average** of the window under it. **(Click to advance.)**

Here the kernel is a *Gaussian blur* (center weighted most, divided by 16), so the patch comes out *smoothed* on the same scale. Cells are shaded by value - light is high.

Convolution: slide, multiply, sum

step 3 of 9

input (5x5, shaded by value)

1	2	3	2	1
2	5	7	5	2
3	7	9	7	3
2	5	7	5	2
1	2	3	2	1

kernel: Gaussian blur / 16

1	2	1
2	4	2
1	2	1

output(0,2) =

weighted average of
the 9 window values

= 4.5

output (smoothed)

4.5	5.6	4.5

Convolution, step by step

Watch the kernel slide. At each stop the output cell is a **weighted average** of the window under it. **(Click to advance.)**

Here the kernel is a *Gaussian blur* (center weighted most, divided by 16), so the patch comes out *smoothed* on the same scale. Cells are shaded by value - light is high.

Convolution: slide, multiply, sum

step 4 of 9

input (5x5, shaded by value)

1	2	3	2	1
2	5	7	5	2
3	7	9	7	3
2	5	7	5	2
1	2	3	2	1

kernel: Gaussian blur / 16

1	2	1
2	4	2
1	2	1

output(1,0) =

weighted average of
the 9 window values

= 5.6

output (smoothed)

4.5	5.6	4.5
5.6		

Convolution, step by step

Watch the kernel slide. At each stop the output cell is a **weighted average** of the window under it. **(Click to advance.)**

Here the kernel is a *Gaussian blur* (center weighted most, divided by 16), so the patch comes out *smoothed* on the same scale. Cells are shaded by value - light is high.

Convolution: slide, multiply, sum

step 5 of 9

input (5x5, shaded by value)

1	2	3	2	1
2	5	7	5	2
3	7	9	7	3
2	5	7	5	2
1	2	3	2	1

kernel: Gaussian blur / 16

1	2	1
2	4	2
1	2	1

output(1,1) =

weighted average of
the 9 window values

= 7

output (smoothed)

4.5	5.6	4.5
5.6	7	

Convolution, step by step

Watch the kernel slide. At each stop the output cell is a **weighted average** of the window under it. **(Click to advance.)**

Here the kernel is a *Gaussian blur* (center weighted most, divided by 16), so the patch comes out *smoothed* on the same scale. Cells are shaded by value - light is high.

Convolution: slide, multiply, sum

step 6 of 9

input (5x5, shaded by value)

1	2	3	2	1
2	5	7	5	2
3	7	9	7	3
2	5	7	5	2
1	2	3	2	1

kernel: Gaussian blur / 16

1	2	1
2	4	2
1	2	1

output(1,2) =

weighted average of
the 9 window values

= 5.6

output (smoothed)

4.5	5.6	4.5
5.6	7	5.6

Convolution, step by step

Watch the kernel slide. At each stop the output cell is a **weighted average** of the window under it. **(Click to advance.)**

Here the kernel is a *Gaussian blur* (center weighted most, divided by 16), so the patch comes out *smoothed* on the same scale. Cells are shaded by value - light is high.

Convolution: slide, multiply, sum

step 7 of 9

input (5x5, shaded by value)

1	2	3	2	1
2	5	7	5	2
3	7	9	7	3
2	5	7	5	2
1	2	3	2	1

kernel: Gaussian blur / 16

1	2	1
2	4	2
1	2	1

output(2,0) =

weighted average of
the 9 window values

= 4.5

output (smoothed)

4.5	5.6	4.5
5.6	7	5.6
4.5		

Convolution, step by step

Watch the kernel slide. At each stop the output cell is a **weighted average** of the window under it. **(Click to advance.)**

Here the kernel is a *Gaussian blur* (center weighted most, divided by 16), so the patch comes out *smoothed* on the same scale. Cells are shaded by value - light is high.

Convolution: slide, multiply, sum

step 8 of 9

input (5x5, shaded by value)

1	2	3	2	1
2	5	7	5	2
3	7	9	7	3
2	5	7	5	2
1	2	3	2	1

kernel: Gaussian blur / 16

1	2	1
2	4	2
1	2	1

output(2,1) =

weighted average of
the 9 window values

= 5.6

output (smoothed)

4.5	5.6	4.5
5.6	7	5.6
4.5	5.6	

Convolution, step by step

Watch the kernel slide. At each stop the output cell is a **weighted average** of the window under it. **(Click to advance.)**

Here the kernel is a *Gaussian blur* (center weighted most, divided by 16), so the patch comes out *smoothed* on the same scale. Cells are shaded by value - light is high.

Convolution: slide, multiply, sum

step 9 of 9

input (5x5, shaded by value)

1	2	3	2	1
2	5	7	5	2
3	7	9	7	3
2	5	7	5	2
1	2	3	2	1

kernel: Gaussian blur / 16

1	2	1
2	4	2
1	2	1

output(2,2) =

weighted average of
the 9 window values

= 4.5

output (smoothed)

4.5	5.6	4.5
5.6	7	5.6
4.5	5.6	4.5

The kernel zoo: hand-designed filters

The same 3×3
machinery, seven classic
kernels. **Predict** before
you look:

$$\text{box blur} = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$\text{Sobel X} = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

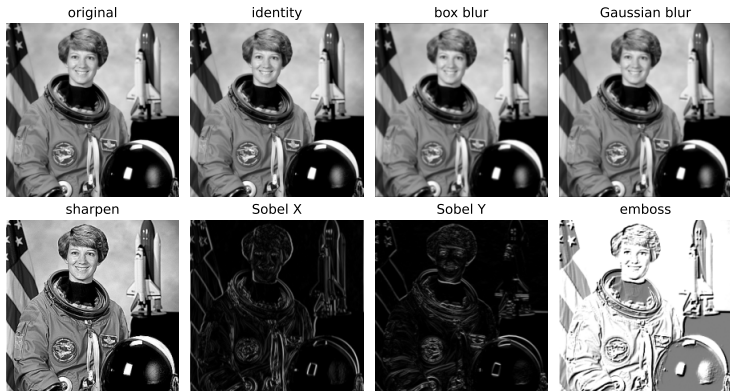
The kernel zoo: hand-designed filters

The same 3×3 machinery, seven classic kernels. **Predict** before you look:

$$\text{box blur} = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$\text{Sobel X} = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Box blur averages a neighborhood; Sobel X responds to *vertical* edges. Same operation, different numbers.



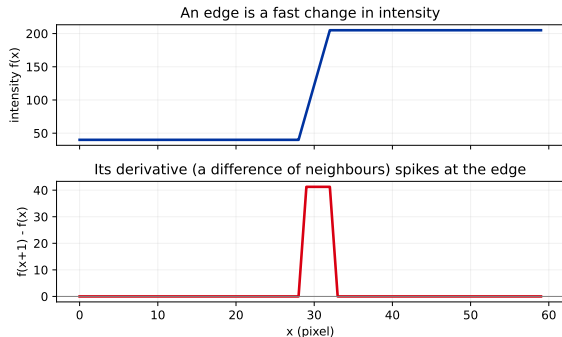
Why do those numbers find edges?

An edge is where intensity **changes fast** - a jump in f . "Fast change" is precisely what a **derivative** measures.

On a pixel grid the derivative is a **difference of neighbours**:

$$f'(x) \approx \frac{f(x+1) - f(x)}{1},$$

which is just $f(x+1) - f(x)$: near zero on flat regions, large at an edge.



The simplest edge detector is a kernel

A *centered* difference looks one step each way:

$$f(x+1) - f(x-1) \iff \text{kernel } [-1 \ 0 \ 1].$$

Slide $[-1 \ 0 \ 1]$ across a row and you read off its horizontal derivative - large wherever a **vertical** edge sits. Its transpose finds horizontal edges.

An edge detector is just a tiny kernel. The calculus hands us the numbers - no guessing required.

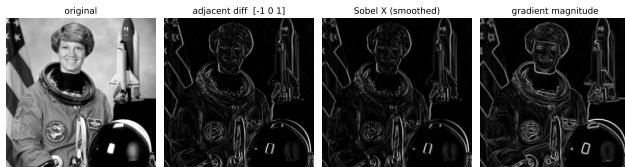
From a difference to Sobel

A bare $[-1 \ 0 \ 1]$ is **noisy** - it fires on every speckle. Fix it by **averaging** down the column while differencing across the row:

$$\underbrace{\begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}}_{\text{smooth}} \underbrace{\begin{bmatrix} -1 & 0 & 1 \end{bmatrix}}_{\text{difference}} = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

That is **Sobel X**; its transpose is Sobel Y. Together they give the gradient - edge strength and direction.

A **spatially separable** kernel (column $\times$ row - two 1D passes, not one 2D). Filters like these ran computer vision for 30 years, each hand-designed by an expert. Hold that thought.



The punchline: a CNN designs its own kernels

A kernel is just a grid of numbers. Those numbers are **weights**.

So instead of an expert choosing $\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$, we let **gradient descent** choose every entry - exactly as it has tuned every weight since [02]. The loss decides which detectors are worth having.

Misconception, pre-empted. CNN kernels are *not* hand-engineered. They are *learned*. That is the whole point: we stop hand-designing feature detectors ([26]) and let the network learn its own representation ([41_1]).

A kernel is a neuron you already know

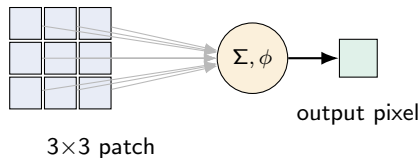
A 3×3 kernel has 9 **weights** + 1 **bias**. At each position it computes

$$z = \sum_k w_k x_k + b, \quad a = \phi(z)$$

an affine map followed by an activation - **exactly the [41_1] neuron**.

Two differences only: it is wired to a small *patch*, not the whole image; and the *same* neuron is reused at every location.

Nothing about training changes. One neuron, reused at every pixel.



Footnote: convolution vs cross-correlation

Purists note: true *convolution* flips the kernel before the dot product; what we (and every deep-learning framework) just did, *without* flipping, is technically **cross-correlation**.

PyTorch's `nn.Conv2d` computes cross-correlation and calls it convolution.

It makes no difference in practice: the kernel is *learned* either way, so it would just learn the flipped version.

Why convolution is the right prior

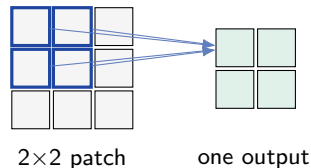
Locality and reuse do not just feel right - they collapse the parameter count.

Sparse connectivity

A 3×3 input, a 2×2 kernel, a 2×2 output. Each output pixel wires only to the 2×2 patch it came from - its **receptive field**.

Count the links: 4 outputs $\times$ 4 inputs each = **16**. A dense layer would wire every output to all 9 inputs: $9 \times 4 = \mathbf{36}$.

Locality alone already cuts the connections.
And it scales: bigger images, far bigger savings.

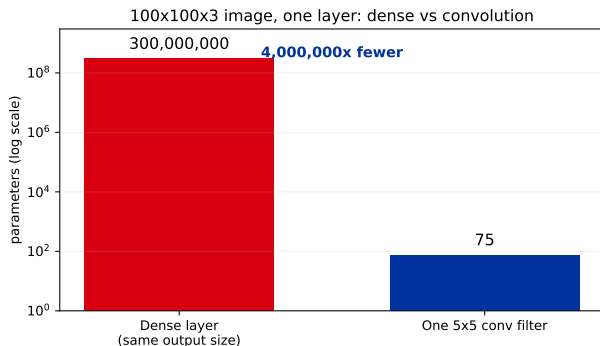


Parameter sharing

Those 16 connections do not use 16 different weights. Every output pixel uses the *same* 4 kernel weights - the kernel is **shared** across all positions.

So: 4 distinct weights, not 36. Scale it up - a $100 \times 100 \times 3$ image, one 5×5 filter, same padding:

$$\underbrace{5 \cdot 5 \cdot 3}_{75} \text{ vs } \underbrace{300,000,000}_{\text{dense}}$$



Regularization by architecture

Fewer parameters plus the locality assumption means a **restricted hypothesis space** - one shaped to match how images actually work. Concretely: a dense layer *can* act as any conv layer (zero most weights, tie the rest), but a conv layer cannot act as most dense layers. The CNN searches a tiny, image-shaped corner of the MLP's space.

- ▶ Weight decay ([43]) *shrinks* weights toward zero.
- ▶ Convolution *removes* them outright, and ties the survivors together.

The strongest regularizer is not a penalty - it is the **right inductive bias**. Trees win on tabular data (ch4, and [40]'s tabular caveat) for the same reason CNNs win on images: the model's assumptions match the data's structure.

The components of a conv layer

Four knobs turn the sliding dot product into a real layer: channels, padding, stride, pooling.

Channels, done properly

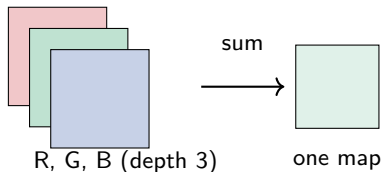
Color returns: an RGB input is a **depth-3 tensor**. One filter spans the *full* depth and **sums across channels** - it outputs *one* feature map, not three.

$$\text{Patch } R = \begin{bmatrix} 1 & 0 \\ 2 & 1 \end{bmatrix}, G = \begin{bmatrix} 0 & 3 \\ 1 & 1 \end{bmatrix}; \quad \text{filter } R = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix},$$

$$G = \begin{bmatrix} 2 & 1 \\ 0 & 0 \end{bmatrix}$$

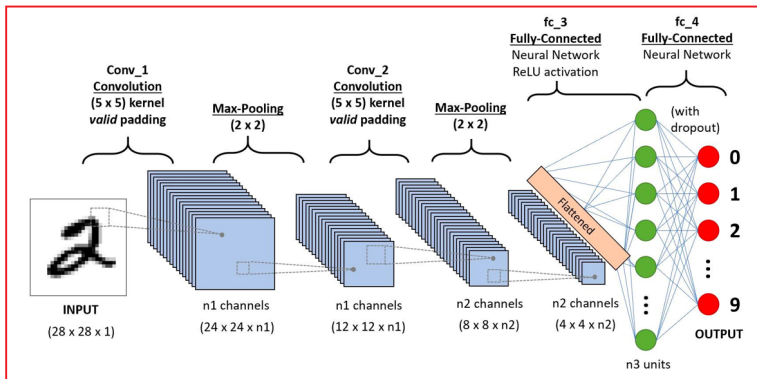
$$\underbrace{(1 \cdot 1 + 0 \cdot 0 + 2 \cdot 0 + 1 \cdot 1)}_{R: 2} + \underbrace{(0 \cdot 2 + 3 \cdot 1 + 1 \cdot 0 + 1 \cdot 0)}_{G: 3} = 5$$

3 channels *in* does not mean 3 maps *out*. N filters give N maps - which become the next layer's depth- N input.



A layer has many kernels

One kernel gives one map. A real conv layer learns **many** kernels at once - each its own detector - so the layer outputs a **stack** of maps (the next layer's depth).



Conv_1 learns n_1 kernels, Conv_2 learns n_2 - a whole CNN, end to end.

The conv-layer parameter formula

$$\text{params} = k \cdot k \cdot C_{\text{in}} \cdot C_{\text{out}} + C_{\text{out}}$$

(one $k \times k$ filter per output channel, spanning the full input depth, plus one bias each)

Two cases by hand:

- ▶ 3×3 conv, RGB $\rightarrow$ 16 maps: $3 \cdot 3 \cdot 3 \cdot 16 + 16 = \mathbf{448}$.
- ▶ the Section-3 5×5 filter on RGB: $5 \cdot 5 \cdot 3 \cdot 1 + 1 = \mathbf{76}$.

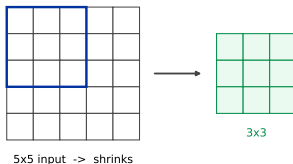
This formula and the output-size formula (next) are the **two tools** for reading any architecture - we use them all through L17 and the homework.

Padding: valid vs same

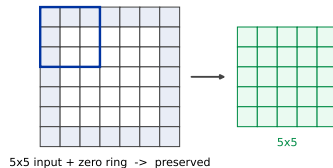
Without padding, every conv *shrinks* the map - so depth is capped: eventually there is nothing left to convolve.

Pad the border with zeros ("same" padding) and the output keeps the input's size. Depth becomes free.

Valid: no padding ($k=3$, $p=0$, $s=1$)



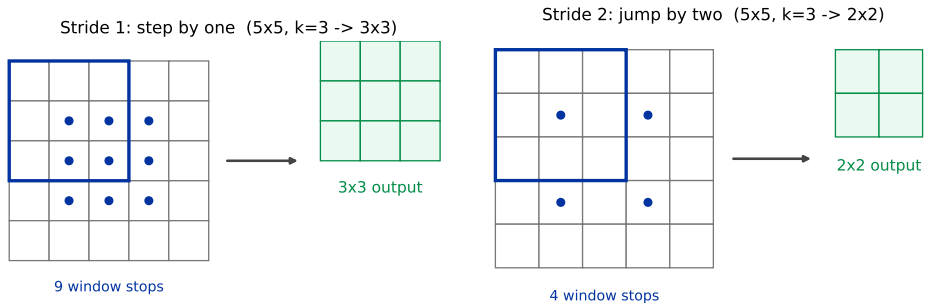
Same: zero-pad 1 ($k=3$, $p=1$, $s=1$)



"valid" = no padding (shrinks).
"same" = pad to preserve size.

What stride controls

Stride s is how far the window jumps between stops. Bigger stride = fewer stops = smaller output (and cheaper).



Stride and the output-size formula

Stride s is how far the window jumps each step. Larger stride = fewer stops = smaller output.

$$o = \left\lfloor \frac{i - k + 2p}{s} \right\rfloor + 1$$

The classic exam question, three cases:

- ▶ $i=28, k=3, p=1, s=1 : \lfloor (28 - 3 + 2)/1 \rfloor + 1 = \mathbf{28}$ (same).
- ▶ $i=28, k=5, p=0, s=1 : \lfloor (28 - 5)/1 \rfloor + 1 = \mathbf{24}$.
- ▶ $i=32, k=3, p=0, s=2 : \lfloor (32 - 3)/2 \rfloor + 1 = \mathbf{15}$.

Pooling

Pooling downsamples each map with a small window and **no weights**. 2×2 windows, stride 2:

1	3
5	6

max = 6

avg = 3.75

2	4
1	2

max = 4

avg = 2.25

7	8
3	2

max = 8

avg = 5

0	1
4	0

max = 4

avg = 1.25

Max keeps the strongest response in each window; **average** smooths. Pooling downsamples, cheapens later layers, and buys a little translation invariance.

Variations on the kernel

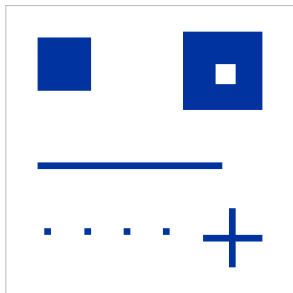
Same sliding window, three twists: a nonlinear cousin, a kernel with gaps, and a kernel that grows the image instead of shrinking it.

Dilation and erosion: kernels that are not sums

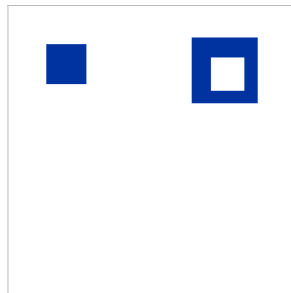
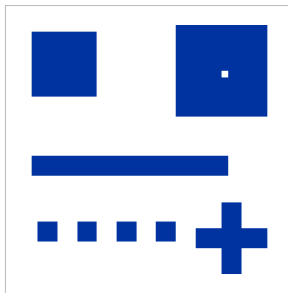
Not every neighborhood operation is a weighted sum. **Morphology** takes the **max** or the **min** over the window:

Morphology: neighborhood max vs min - opposites (duals), not weighted-sum convolution

Original



Dilation (3x3 max): grows, fills holes Erosion (3x3 min): shrinks, breaks thin parts

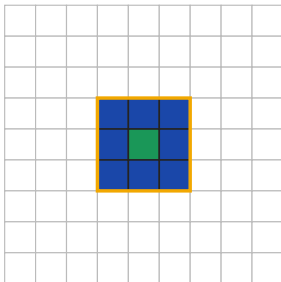


Dilation = max (grows bright regions, fills small holes); **erosion** = min (shrinks them, removes thin bits). They are exact **opposites** (duals). Classic pre-CNN tools for cleaning up masks - but *nonlinear*, so there are no learned weights here.

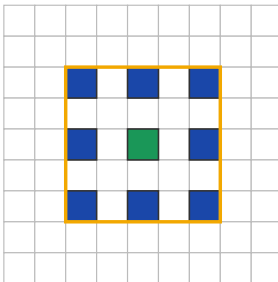
Dilated convolution: a kernel with gaps

Dilated convolution: gaps in the kernel widen the receptive field for free (green = center tap)

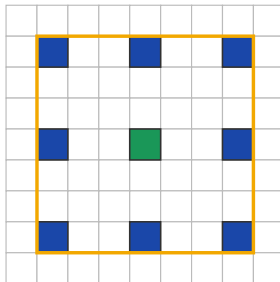
dilation $d = 1$
9 weights, sees 3×3



dilation $d = 2$
9 weights, sees 5×5



dilation $d = 3$
9 weights, sees 7×7

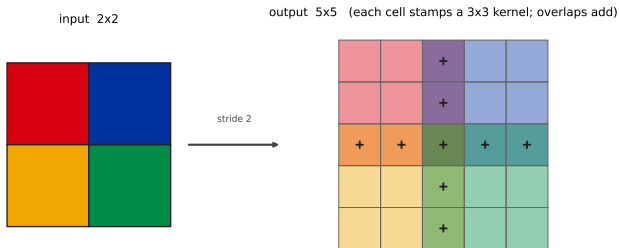


Space the kernel taps d **apart**: the same 9 weights now cover a $(2d+1) \times (2d+1)$ patch - a **wider receptive field**, **no extra parameters**, **no pooling**.

Stacking layers also widens the view (L17); dilation does it *within one layer*. Back in **L19** for segmentation: wide context, every pixel kept.

Transposed convolution: run it backwards to upsample

Transposed convolution: the opposite of a shrinking conv - it upsamples (2x2 -> 5x5)



A normal conv **shrinks** the map; a **transposed** conv **grows** it - each input cell *stamps* the kernel onto a larger output, and overlaps add. In 1D: input (1, 2), kernel (1, 1, 1), stride 2 stamps (1, 1, 1) and (2, 2, 2) two apart → (1, 1, **3**, 2, 2) - the 3 is the overlap.

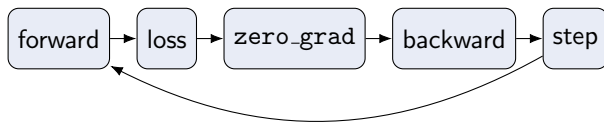
The tool for when the **output must be an image** - upsampling in **L19's** U-Net decoder and GAN generators. Watch for checkerboard artifacts.

Your first CNN

Stack the blocks, track the tensor, and let autograd do the rest.

The training loop, in 60 seconds (recall [42])

Nothing about training changes for a CNN. The loop from [42] and your ch11 homework, unchanged:



Forward pass, compute loss, clear old gradients, backprop, update. The only new thing today is *which layers* sit inside the forward pass.

Training a shared kernel: the gradients add up

A kernel weight is used at every position - so which position updates it? **All of them.** [42]'s chain rule already said so: when a variable feeds several outputs, its gradient is the **sum** over them, $\frac{\partial z}{\partial x_i} = \sum_j \frac{\partial z}{\partial y_j} \frac{\partial y_j}{\partial x_i}$.

1D, by hand: input $\mathbf{x} = (1, 2, 0, 3, 1)$, kernel (w_1, w_2, w_3) , so $y_j = w_1 x_j + w_2 x_{j+1} + w_3 x_{j+2}$ for $j = 1, 2, 3$. Backprop hands us $g_j = \partial L / \partial y_j = (0.5, -1, 2)$. Then

$$\frac{\partial L}{\partial w_1} = \sum_j g_j x_j = 0.5 \cdot 1 - 1 \cdot 2 + 2 \cdot 0 = -1.5$$

$$\frac{\partial L}{\partial w_2} = \sum_j g_j x_{j+1} = 0.5 \cdot 2 - 1 \cdot 0 + 2 \cdot 3 = 7$$

(w_2 met x_2, x_3, x_4 - shift by one.) Same way, $\partial L / \partial w_3 = -1$.

The pattern: $\partial L / \partial w_k = \sum_j g_j x_{j+k-1}$ - slide $\mathbf{g}$ along $\mathbf{x}$, take dot products. **The gradient of a convolution is itself a convolution.** In 2D the same, over a 2D window.

Sharing helps learning: every kernel weight collects a vote from every position of every image in the batch.

Anatomy of a conv stack

[conv $\rightarrow$ ReLU $\rightarrow$ pool] $\times 2 \rightarrow$ flatten $\rightarrow$ dense $\rightarrow$ softmax, a 28×28 input:

Layer	Output shape	Params
input	$1 \times 28 \times 28$	0
conv $1 \rightarrow 8$, 3×3 , pad 1	$8 \times 28 \times 28$	80
ReLU	$8 \times 28 \times 28$	0
maxpool 2	$8 \times 14 \times 14$	0
conv $8 \rightarrow 16$, 3×3 , pad 1	$16 \times 14 \times 14$	1,168
ReLU	$16 \times 14 \times 14$	0
maxpool 2	$16 \times 7 \times 7$	0
flatten	784	0
dense $784 \rightarrow 10$	10	7,850
total		9,098

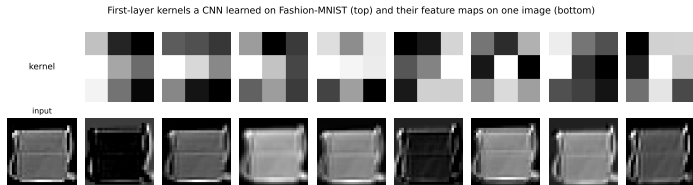
Two things to notice: the **ReLU** matters - without it, stacked convs collapse into one big conv ([41_1]'s linearity lesson); and **pooling has zero parameters**.

What the layers learn

Train that CNN on Fashion-MNIST and look at its first-layer kernels: edge and texture detectors *nobody designed*.

Deeper layers compose these into parts, then objects.
(Trained on ImageNet, first-layer kernels look like textbook edge detectors - L17.)

This is [41_1]'s XOR lesson at scale: the network re-coordinatises pixels until a plain linear head can separate the classes.



The canonical snippet

The architecture from the anatomy slide, in PyTorch:

```
import torch.nn as nn

net = nn.Sequential(
    nn.Conv2d(1, 8, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
    nn.Conv2d(8, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
    nn.Flatten(),
    nn.Linear(16 * 7 * 7, 10),
)
# training loop: unchanged from the ch11 homework
# forward -> loss -> zero_grad -> backward -> step
```

Autograd differentiates a conv like anything else - [42]'s `loss.backward()` promise (one backward sweep, any graph), kept.

Why this is fast

A convolution unrolls into a large **matrix multiply**: every output pixel is a dot product, and all of them stack into one big matmul.

That is exactly the shape a **GPU** eats ([42]'s hardware pages) - thousands of identical multiply-adds in parallel.

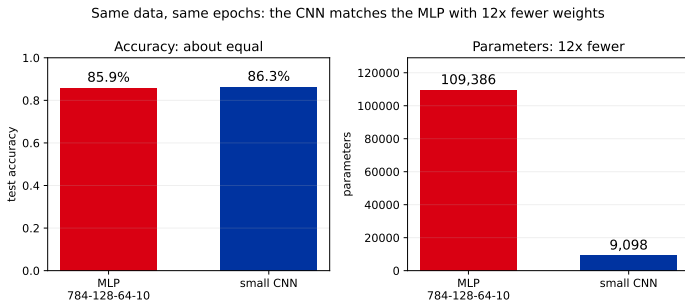
Same reason AlexNet's 60 million parameters trained in under a week on two gaming GPUs in 2012 (next lecture): the operation maps perfectly onto parallel hardware.

Payoff: CNN vs MLP, same data

Fashion-MNIST (the ch11 homework's data), 15k training images, 15 epochs each: a 784-128-64-10 MLP vs this small CNN.

Accuracy is **about equal**, but the CNN uses $12\times$ **fewer** weights.

The *architecture*, not raw capacity, did the work. On color photos (CIFAR-10, HW2) the gap becomes a chasm - measure it yourself.



Recap

- ▶ A dense net cannot afford pixels and ignores their structure.
- ▶ Convolution = a **local, shared** neuron - locality and weight reuse, built in (the same two tricks as the visual cortex).
- ▶ One filter spans the *full input depth* and outputs *one* map; N filters $\rightarrow N$ maps.
- ▶ Two formulas run everything: $o = \lfloor (i - k + 2p)/s \rfloor + 1$ and $\text{params} = k \times k \times C_{\text{in}} \times C_{\text{out}} + C_{\text{out}}$.
- ▶ Pooling has **zero parameters**; it downsamples and buys a little invariance.
- ▶ Kernels are **learned**, not hand-designed - a shared weight's gradient is the **sum** over every position it touched.

Next: we can build a small CNN. What did 10 years of architecture design buy? The ImageNet story - from 26% error to superhuman (L17).